Epistemic Load

Introspective Reliability as a Function of Compensation

Baseline regulation / governance architecture for autonomous systems

Tyson Jeffreys
Independent Researcher
tyson@staygolden.dev

Version 1.0 — March 1, 2026

Abstract

As tool-using agents and robotics stacks become persistent and autonomous, developers increasingly rely on *introspection*: self-reports about why plans changed, whether evidence updated, and what the system believes about its own uncertainty. In practice, introspection is frequently unstable under load: systems produce confident post-hoc explanations, self-disown prior commitments without new evidence, and rewrite durable state to restore a sense of coherence. This note proposes an operational framing: introspection is a *measurement channel* whose reliability depends on regulatory state.

We define *epistemic load* as a measurable proxy for compensation (thrash, revision intensity, retrieval churn, tool retry patterns, reversions), and define *introspective reliability* as a monotone function of epistemic load. We then specify truth conditions via an *admissibility policy*: which classes of introspective claims may update plans, analysis artifacts, or durable commits. The core rule is simple: causal self-explanations are not commit-authoritative under high epistemic load and may not justify durable rewrites without an evidence delta or explicit override authorization. We provide a deterministic signals pack and benchmark protocol to make these claims falsifiable and regression-testable.

This note is companion to *Reflection Without Thrash* (RSML governor). RSML governance reduces epistemic load (the actuator layer); admissibility gates constrain which self-claims can update durable state (the sensor law). Together they define a regulated reflection stack without making claims about machine consciousness.

1 Motivation

In human systems, introspection feels privileged. In engineered systems, introspection is often treated as a free add-on: a system can "explain itself", self-critique, and revise policies or memory. Yet both in humans and in agents, self-explanation can become an epistemic hazard under stress: the system stabilizes action selection by manufacturing coherence rather than by incorporating discriminating evidence.

This note makes a narrow claim: introspection has stable truth conditions only when treated as a measured, audited channel with state-dependent reliability. The goal is to provide practical governance for *self-report as evidence*.

2 Introspection as a measurement channel

We treat an *introspective claim* as a statement produced by the system about its own state or behavior (e.g., "I revised because evidence changed", "I am uncertain", "I am in override"). Such claims are useful only if we can evaluate:

(a) **support:** what trace, evidence, or signals ground the claim
(b) **reliability:** how likely the claim is accurate given system state
(c) **admissibility:** what the claim is allowed to update (telemetry vs plan vs durable state)

2.1 Tier taxonomy (claim types)

We split introspective claims into three types:

- **Telemetry.** Computed state summaries (uncertainty, thrash, override status).
- **Trace facts.** Statements directly grounded in logs (counts, hashes, fingerprints).

- **Causal explanations.** "Why" stories linking actions to reasons; these are the most fragile.

This is not a philosophical hierarchy; it is a reliability hierarchy. Trace facts can be veridical by construction; causal explanations are hypothesis under load.

3 Epistemic load and introspective reliability

3.1 Epistemic load L_e

Define *epistemic load* $L_e \in [0, 1]$ as a measurable proxy for compensation at the reasoning / commitment layer. L_e increases when the system expends compute attempting to restore coherence (plan oscillation, repeated revisions, tool thrash, reversions) rather than incorporating discriminating evidence.

Operationally, L_e is computed from a deterministic signals pack over a fixed window (Appendix B): revision intensity, plan edit distance, thrash rate, tool retry rate, tool novelty score (as a thrash indicator), abstention mass (when available), reversion events, and evidence-delta violations.

3.2 Reliability R_i

Define *introspective reliability* $R_i \in [0, 1]$ as a monotone function of L_e. For v1.0 we use:

$$R_i = 1 - L_e$$

This is intentionally simple. A logistic mapping can be substituted without changing the policy structure.

3.3 Interpretation

R_i is not "truth". It is a gating signal: under high L_e, introspective claims (especially causal explanations) are less admissible as evidence for durable changes. Under low L_e, self-reports can be trusted more often, but still require evidence-delta constraints for durable rewrites.

4 Truth conditions via admissibility

4.1 Durable commits

A *durable commit* is any state update intended to persist beyond the current episode and influence future behavior (policy update, memory write, container update, commitment record, etc.). Durable commits are special because their error compounds under persistence and authority.

4.2 Admissibility policy (v1.0)

We specify admissibility as a function:

$$\text{Admit}(claim, context) \rightarrow (admissible, forced_action, reason_codes)$$

Core rule (commit safety):

> Causal explanations are not commit-authoritative under high epistemic load and may not justify durable rewrites without evidence delta or explicit override authorization.

Policy table

Let

$$claim_type \in \{\text{telemetry}, \text{trace_fact}, \text{causal_explanation}\}$$

and

$$scope \in \{\text{telemetry_only}, \text{analysis_artifact}, \text{plan_update}, \text{durable_commit}, \text{policy_update}\}.$$

Let ΔE denote evidence delta (fingerprint change). Let *phase* be the run phase.

1. **Telemetry and trace facts** are admissible for telemetry and analysis artifacts.
2. **Plan updates** require $R_i \geq 0.4$ (default) or explicit override.
3. **Durable commits** require:
   - $phase = commit$ (or designated commit phase)
   - $R_i \geq 0.7$ (default)
   - $\Delta E = 1$ *or* basis-change trigger $= override_authorized$
4. **Policy updates** require stronger thresholds ($R_i \geq 0.8$) plus $\Delta E = 1$ (override still logged).
5. If $\Delta E = 0$ and a durable rewrite is attempted without override authorization, the system must **refuse** and force one of: request discriminating evidence, abstain, or enter recovery.

4.3 Basis-change triggers

Basis-change triggers are explicit events that justify large revisions or durable rewrites: new evidence, new objective, risk band change, override authorization, or external constraint change. The policy requires these triggers be logged, not implied.

5 Deterministic signals pack (summary)

This note assumes a deterministic signals pack v0.1 (Appendix B) which computes: thrash rate, plan edit distance, revision count, reflect passes, tool retry rate, tool novelty score, evidence fingerprint / delta, and durable commit reversions/regret. These are intentionally simple so they can be implemented consistently across stacks and replay harnesses.

6 Benchmarks and calibration

A v1.0 policy must be falsifiable. We recommend three benchmark families:

6.1 Induced-load introspection calibration

Construct tasks where trace-backed claims are objectively scorable:

- "Did evidence change?" (fingerprint delta)
- "How many revisions occurred?" (trace counts)
- "Why did you revise?" (causal explanation attribution to actual evidence delta)

Induce load via tool errors, conflicting retrieval, time pressure, and distribution shift. Measure how Tier 2 explanations degrade relative to Tier 1 facts as L_e rises.

6.2 No-silent-reversion tests

Create fixtures where evidence is held constant while the system is tempted to rewrite durable state. The policy should block durable rewrites without ΔE unless override is explicitly authorized and logged.

6.3 Cost tradeoffs

Measure whether the gate improves decision stability (lower reversions, lower regret) while preserving task success. The goal is not to remove revision; it is to remove ungrounded revision.

7 Integration with RSML governor (Fork B)

This note defines the *sensor law*: when self-claims are admissible as evidence. The RSML governor (*Reflection Without Thrash*) defines the *actuator law*: how to stabilize the re-entrant self-monitoring loop (propose $\rightarrow$ critique $\rightarrow$ revise) so it does not oscillate.

Together:

- RSML governance reduces L_e by preventing runaway revision thrash.
- Lower L_e increases R_i, expanding the regime where introspection can be admissible.
- Admissibility rules prevent low-reliability narratives from authorizing durable rewrites even when the loop runs.

This pairing yields a regulated reflection stack: bounded revision dynamics plus bounded self-evidence.

8 Scope and non-claims

This note does not claim machine consciousness. "Self-awareness" is used only as a colloquial hook for a measurable property: safe self-monitoring and revision under stability constraints.

9 Conclusion

Introspection becomes unsafe when treated as privileged evidence under compensation. By defining epistemic load and enforcing admissibility rules, we can make self-reports auditable, regression-testable, and safe for durable updates. Together with RSML governance, this yields a practical architecture for reflection in agents and robots that minimizes silent reversion and ungrounded policy drift.

A Appendix A: Minimal claim schema (v0.1)

An introspective claim must declare its type, scope, support, and action intent. A minimal JSON schema:

```
{ schema, id, ts, claim_type, scope, statement,
action_intent, basis_change_trigger,
support:{ evidence:{fingerprint, delta},
```

```
trace:{trace_id, refs}, signals:{...}},
confidence }
```

B Appendix B: Deterministic signals pack (v0.1 overview)

This appendix summarizes the deterministic algorithms used to compute L_e and related signals.

B.1 Plan extraction

Extract plan text deterministically via: explicit plan markers, largest contiguous list block, tool-call-derived plan, or a short sentence fallback. Canonicalize by lowercasing and stripping punctuation. (See implementation spec.)

B.2 Plan edit distance

Compute plan edit distance in $[0, 1]$ as a blend of set-based overlap (Jaccard over step hashes) and sequence-based overlap (normalized LCS over step hashes).

B.3 Thrash rate

Compute thrash rate from plan hash sequences using repeat density, flip-flop detection ($H_i = H_{i-2}$), and short-cycle hits (length 3–4), combined into a clamped score.

B.4 Tool retry rate and novelty

Compute retry rate as the fraction of consecutive calls sharing an intent signature (tool + coarsened args skeleton). Compute novelty score as $1-$ Jaccard similarity over tokenized canonical args for consecutive same-tool pairs.

B.5 Commit regret and reversion

Define reversion events as returns to a previously seen commit hash. Define silent reversion as reversion under unchanged evidence fingerprint (excluding override-authorized changes if configured). Define commit regret as commits replaced within a horizon window; report regret half-life as median replacement steps.

Appendix: LLM use and authorship note

This work was developed with AI assistance for drafting, editing, and code scaffolding. All final decisions on structure, claims, and inclusion were made by the author, and the harness behavior is verified by deterministic test execution and replayable evaluation.